

# MINERÍA DE DATOS

**Maximiliano Ojeda**

[muojeda@uc.cl](mailto:muojeda@uc.cl)

---



IIC-2433

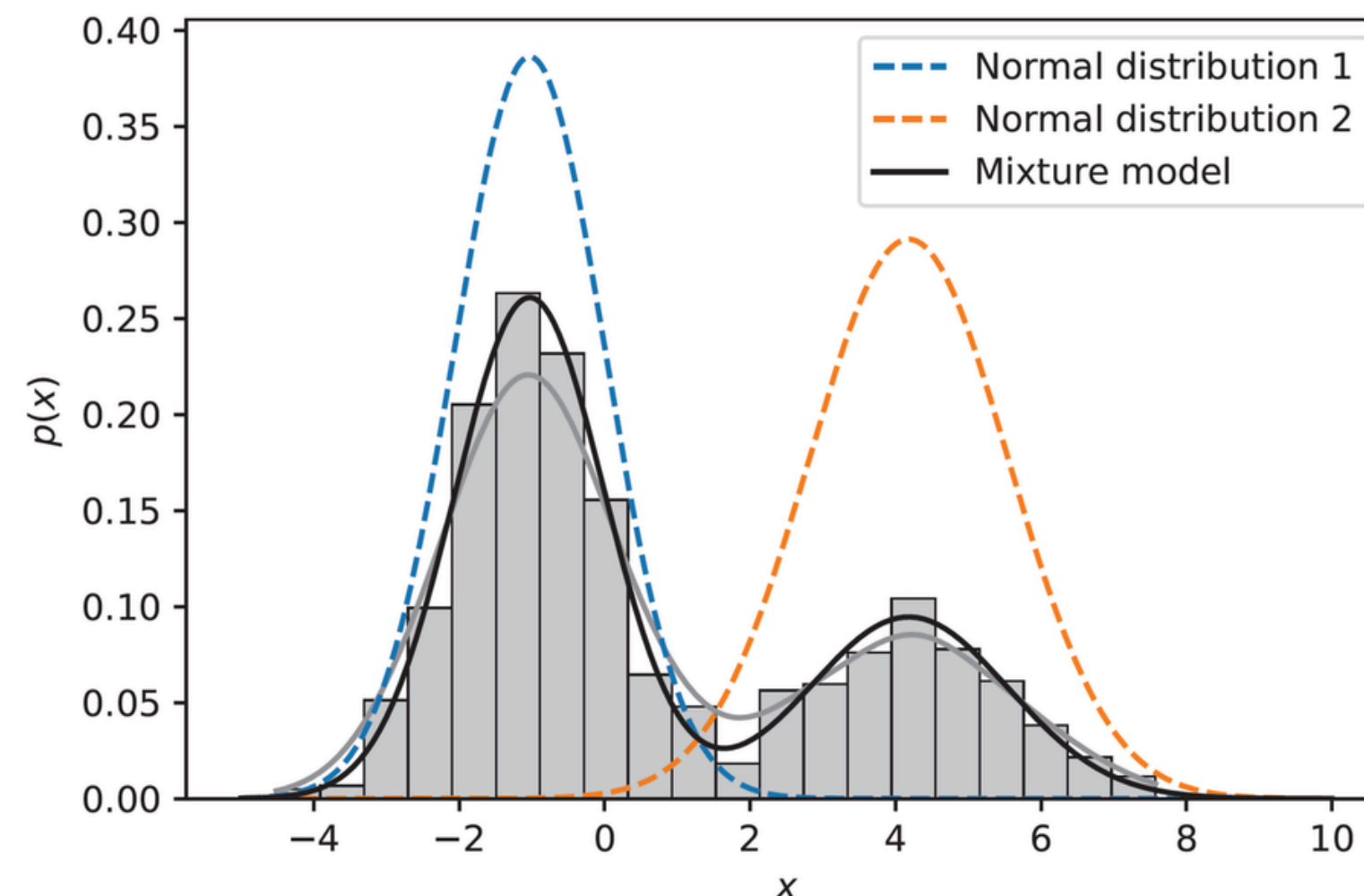


# Gaussian Mixtures Model

# GMM

Es un **modelo probabilístico** que asume que los puntos de datos se generan a partir de una mezcla de varias distribuciones gaussianas con **parámetros desconocidos**.

A diferencia de los métodos de clustering duros como K-Means, que asignan cada punto a un único clúster según el centroide más cercano, el GMM realiza un clustering suave **asignando a cada punto una probabilidad de pertenecer a múltiples clústeres**.



# Recordatorio: Verosimilitud

**La verosimilitud mide qué tan bien un conjunto de parámetros explica los datos que observamos.** Es una función de parámetros para datos fijos.

Si tenemos un modelo probabilístico  $P(X \mid \theta)$

$X = \{x_1, x_2, \dots, x_n\}$  : son los datos observados

$\theta$  : son los parámetros del modelo (por ejemplo, media y varianza)

entonces la **verosimilitud** se define como:

$$L(\theta \mid X) = \prod_{i=1}^n P(x_i \mid \theta)$$

y su versión logarítmica (más usada) es:

$$\log L(\theta \mid X) = \sum_{i=1}^n \log P(x_i \mid \theta)$$

# MLE

La **estimación de máxima verosimilitud** (MLE) es un método estadístico que se utiliza para estimar los parámetros de una distribución de probabilidad mediante **maximizar la función de verosimilitud**.

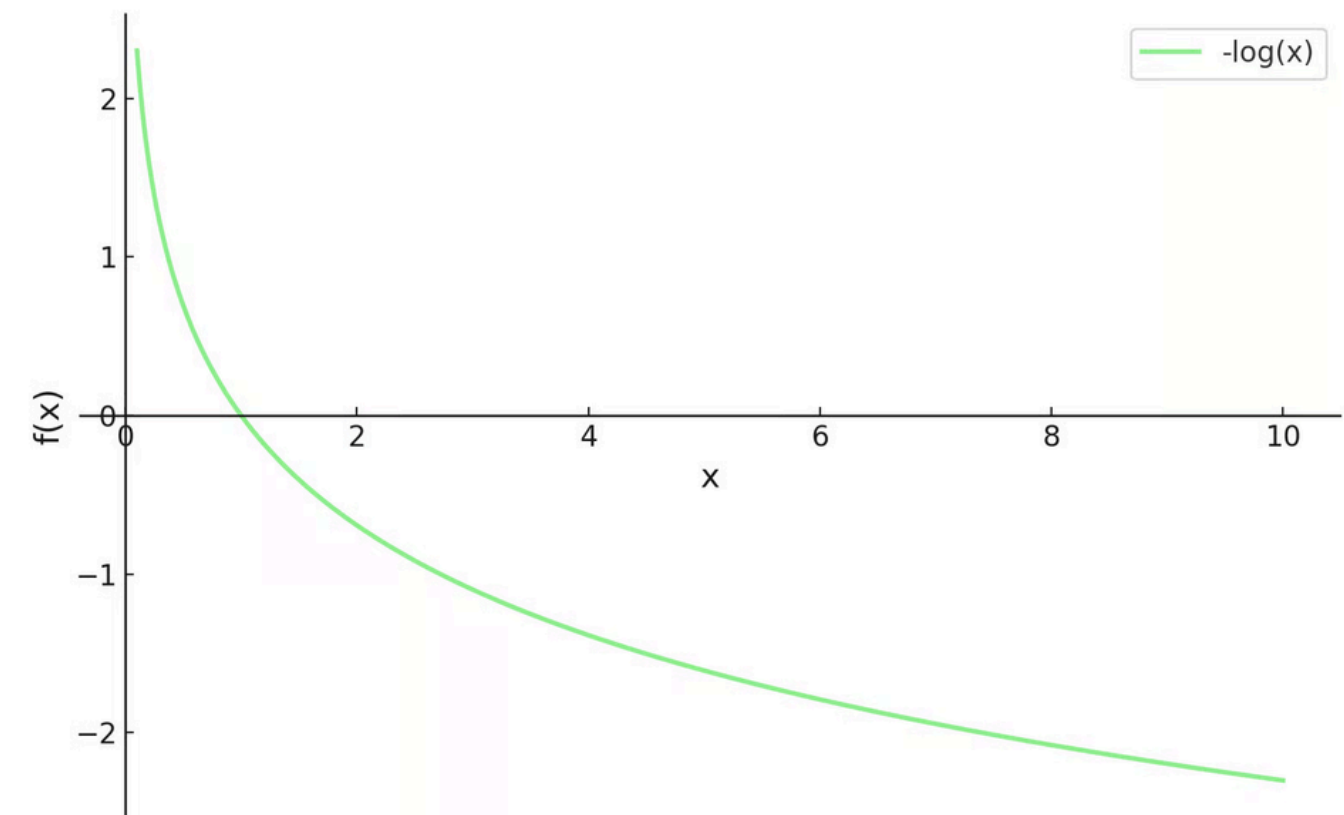
$$\hat{\theta} = \arg \max_{\theta} L(\theta)$$

o bien,

$$\hat{\theta} = \arg \max_{\theta} \sum_{i=1}^n \log P(x_i \mid \theta)$$

en general se suele utilizar la log-verosimilitud negativa

$$\hat{\theta} = \arg \min_{\theta} \left[ - \sum_{i=1}^n \log P(x_i \mid \theta) \right]$$





# Ejemplo: MLE

Supongamos que lanzamos un dado 12 veces y anotamos los resultados. Queremos modelar estos datos utilizando una distribución categórica, pero centrémonos en **estimar la probabilidad  $\theta$  de sacar un seis**. En este ejemplo:

- **Parámetro ( $\theta$ )** El valor que queremos estimar: probabilidad de sacar un seis.
- **Data ( $x$ )** Los resultados que observamos: 4 seis en 12 lanzamientos.

Ahora calculamos la función de verosimilitud, que, dado que obtuvimos 4 seises en 12 lanzamientos, sería:

$$L(\theta) = \theta^4 \times (1 - \theta)^8$$

Tomamos la **log-verosimilitud negativa**, lo que nos da esta ecuación:

$$\ell(\theta) = -4 \ln \theta - 8 \ln(1 - \theta)$$

Derivado la ecuación e igualando a 0:

$$\ell'(\theta) = \frac{8}{1 - \theta} - \frac{4}{\theta} = 0 \quad \Rightarrow \quad \theta = \frac{1}{3}$$

# Mixture Models

Es un modelo probabilístico que se utiliza para representar **datos que pueden provenir de categorías diferentes, cada una de las cuales se modela mediante una distribución de probabilidad distinta.**

Formalmente, si  $X$  es una variable aleatoria cuya distribución es una mezcla de  $K$  distribuciones componentes, la función de densidad de probabilidad de  $X$  puede escribirse como:

$$p(x) = \sum_{k=1}^K w_k f_k(x; \theta_k)$$

$K$  : es el número de distribuciones componentes en la mezcla.

$f_k()$  : es la función de densidad de la distribución componente k-ésima.

$w_k$  : es el peso de mezcla del componente k-ésimo

$\theta_k$  : representa los parámetros del componente k-ésimo

# Gaussian Mixture Models

Es un modelo probabilístico que asume que los puntos de datos se generan a partir de una **mezcla de varias distribuciones gaussianas con parámetros desconocidos**.

$$p(\mathbf{x}) = \sum_{k=1}^K w_k \mathcal{N}(\mathbf{x}; \mu_k, \Sigma_k)$$

$K$  : es el número de distribuciones componentes en la mezcla.

$\mathcal{N}(\mathbf{x}; \mu_k, \Sigma_k)$  : es la función de densidad normal multivariante para el componente k-ésimo

$\Sigma_k$  : es la matriz de covarianza del componente gaussian

$\mu_k$  : es el vector de medias del componente gaussiano

$w_k$  : es el peso de mezcla del componente k-ésimo

$$\mathcal{N}(\mathbf{x}; \mu_k, \Sigma_k) = \frac{1}{\sqrt{(2\pi)^d |\Sigma_k|}} \exp \left( -\frac{1}{2} (\mathbf{x} - \mu_k)^\top \Sigma_k^{-1} (\mathbf{x} - \mu_k) \right)$$



# Aprender Parámetros GMM

El objetivo es **encontrar los parámetros del GMM** (medias, covarianzas y coeficientes de mezcla) que mejor expliquen los datos observados

Verosimilitud,

$$\mathcal{L}(\theta \mid X) = \prod_{i=1}^n \left[ \sum_{k=1}^K w_k \mathcal{N}(x_i; \mu_k, \Sigma_k) \right]$$

Log-Verosimilitud,

$$\ell(\theta \mid X) = \sum_{i=1}^n \log \left[ \sum_{k=1}^K w_k \mathcal{N}(x_i; \mu_k, \Sigma_k) \right]$$

No podemos aplicar directamente MLE para estimar los parámetros de un GMM:

- La función de log-verosimilitud es **altamente no lineal** y compleja de maximizar analíticamente
- El modelo **posee variables latentes** (los pesos de mezcla) que **no son directamente observables en los datos**

# Expectation-Maximization (EM)

Es un método para encontrar las **estimaciones de máxima verosimilitud** de los parámetros en modelos estadísticos que dependen de variables latentes no observadas, como GMM.

El algoritmo comienza inicializando aleatoriamente los parámetros del modelo, y luego itera entre dos pasos :

## 1. Expectación (E-step):

Calcula la **log-verosimilitud esperada** del modelo con respecto a la distribución de las variables latentes, dadas las observaciones y los valores actuales de los parámetros del modelo. En este paso, **se estima la probabilidad de las variables latentes** (la probabilidad de que cada punto pertenezca a cada componente).

## 2. Maximización (M-step):

**Actualiza los parámetros del modelo** para maximizar la log-verosimilitud de los datos observados, utilizando las probabilidades de las variables latentes obtenidas en el paso E.

Estos dos pasos se **repiten hasta la convergencia**, la cual suele determinarse mediante un umbral en el cambio de la log-verosimilitud o tras alcanzar un número máximo de iteraciones.

# Asginación de Clusters

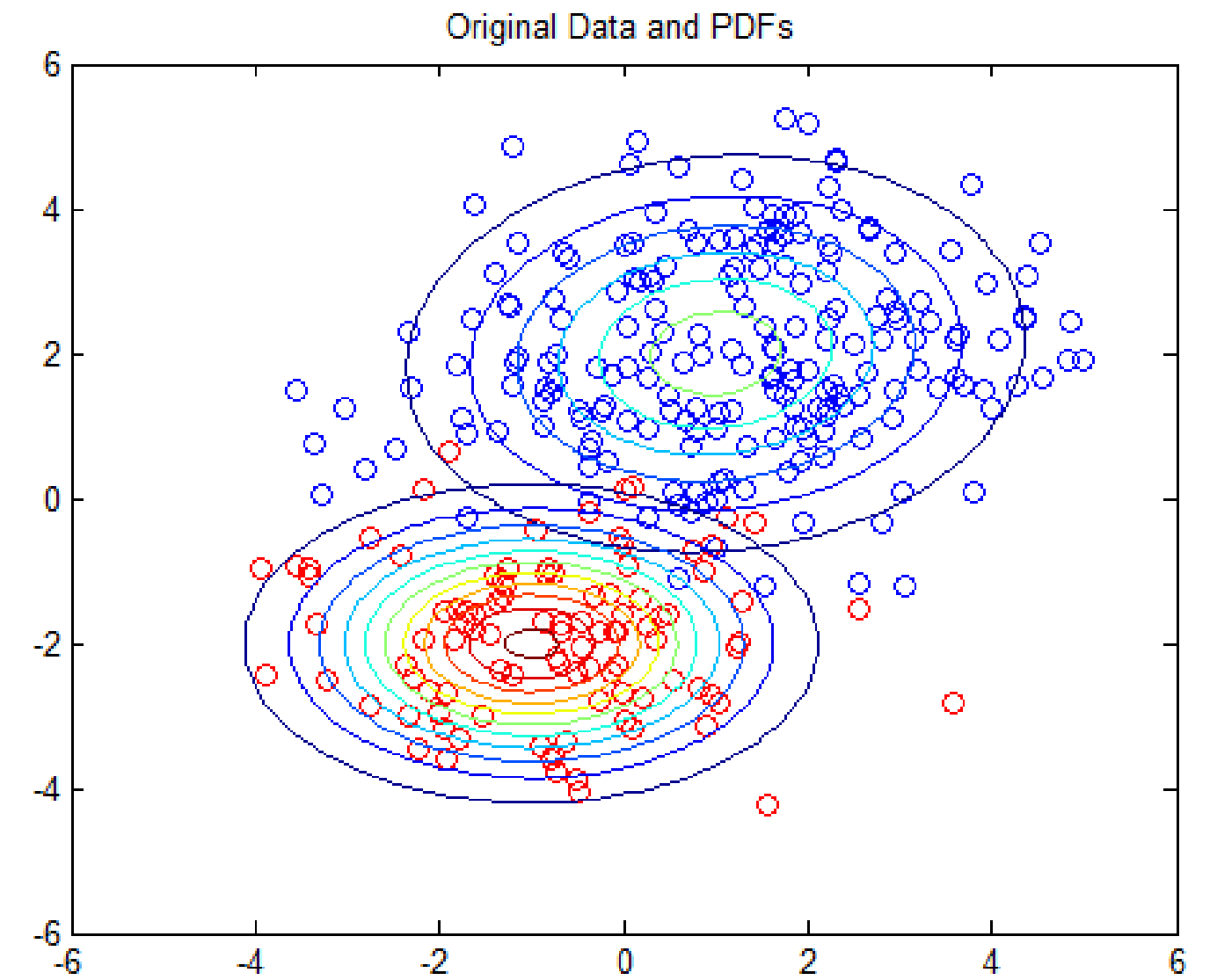
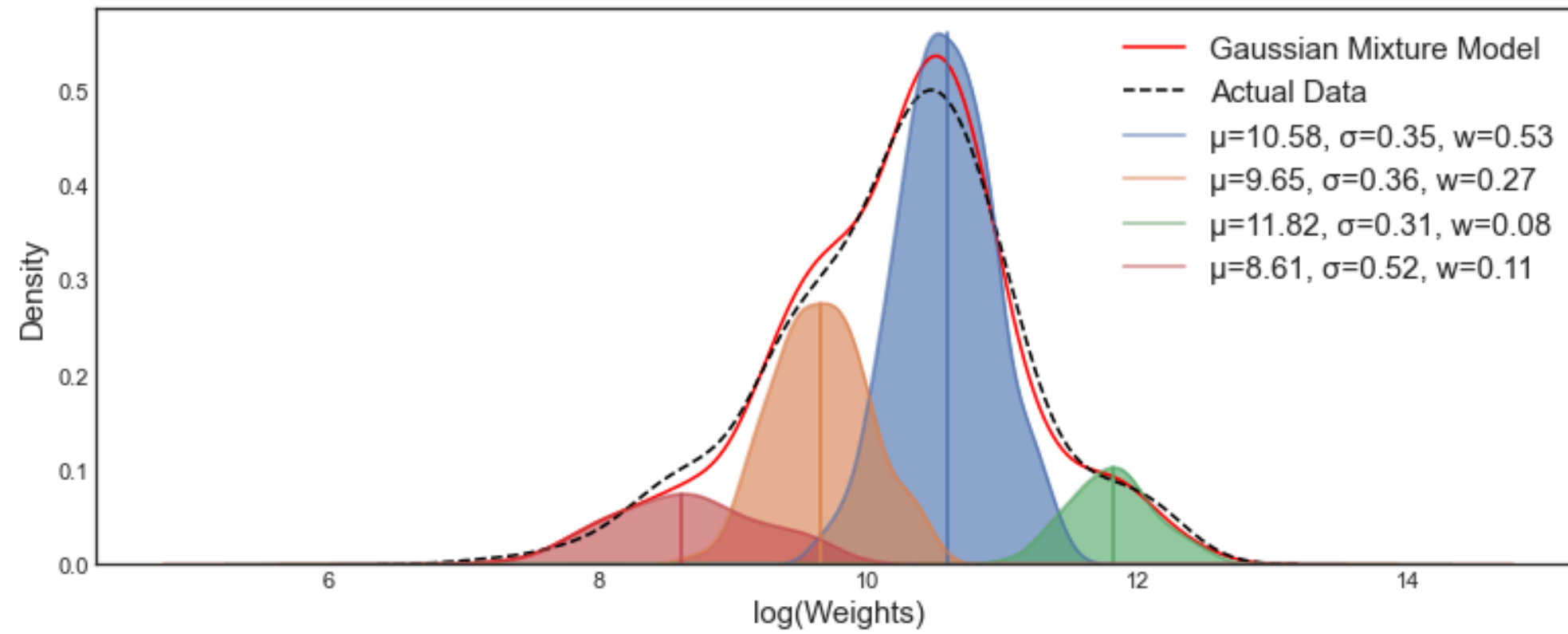
Una vez que el modelo aprende los parámetros de cada gaussiana, puede calcular **la probabilidad de que un punto  $x_i$  haya sido generado por cada componente  $k$ .**

Para cada punto  $x_i$ , se calcula su responsabilidad o probabilidad posterior de pertenecer al clúster  $k$ :

$$\gamma_{ik} = P(k \mid x_i) = \frac{w_k \mathcal{N}(x_i \mid \mu_k, \Sigma_k)}{\sum_{j=1}^K w_j \mathcal{N}(x_i \mid \mu_j, \Sigma_j)}$$

$\gamma_{ik}$  son las probabilidades de pertenencia, es decir, cuánto contribuye cada gaussiana a explicar el punto

# GMM



# GMM

El único parámetro que debemos definir es la cantidad de clusters: K

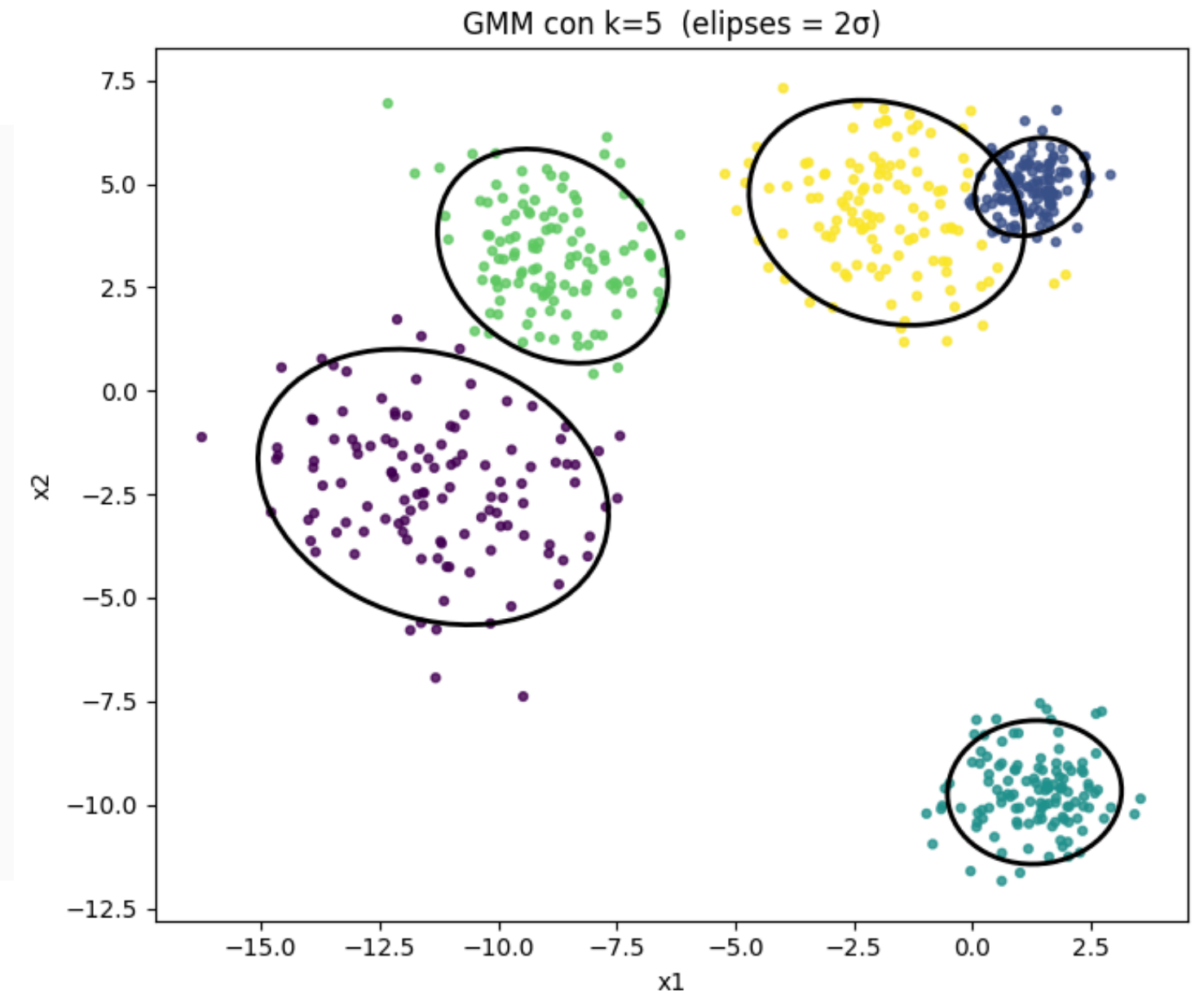
```
from sklearn.mixture import GaussianMixture
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=300, centers=5, cluster_std=[1.3, 0.6, 0.9, 1.9, 1.4])

# Crear y ajustar un modelo GMM con 5 componentes
gmm = GaussianMixture(n_components=5, random_state=42)
gmm.fit(X)

# Predecir a qué componente pertenece cada punto
labels = gmm.predict(X)

plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.title("Clustering con Gaussian Mixture Model (GMM)")
plt.show()
```





# Elección de K

La log-verosimilitud no sirve por sí sola para elegir K: siempre mejora (o se mantiene igual) al agregar más componentes, incluso cuando eso produce sobreajuste.

Necesitamos un criterio que penalice la complejidad del modelo, no solo premie el ajuste a los datos.

**AIC (Akaike Information Criterion):**

$$AIC = 2p - 2\ell(\hat{\theta} \mid X)$$

**BIC (Bayesian Information Criterion):**

$$BIC = p \cdot \ln(n) - 2\ell(\hat{\theta} \mid X)$$

$\ell(\hat{\theta} \mid X)$ : log-verosimilitud máxima alcanzada por el modelo

$p$  : cantidad de parámetros libres del modelo

$n$  : cantidad de datos

# GMM

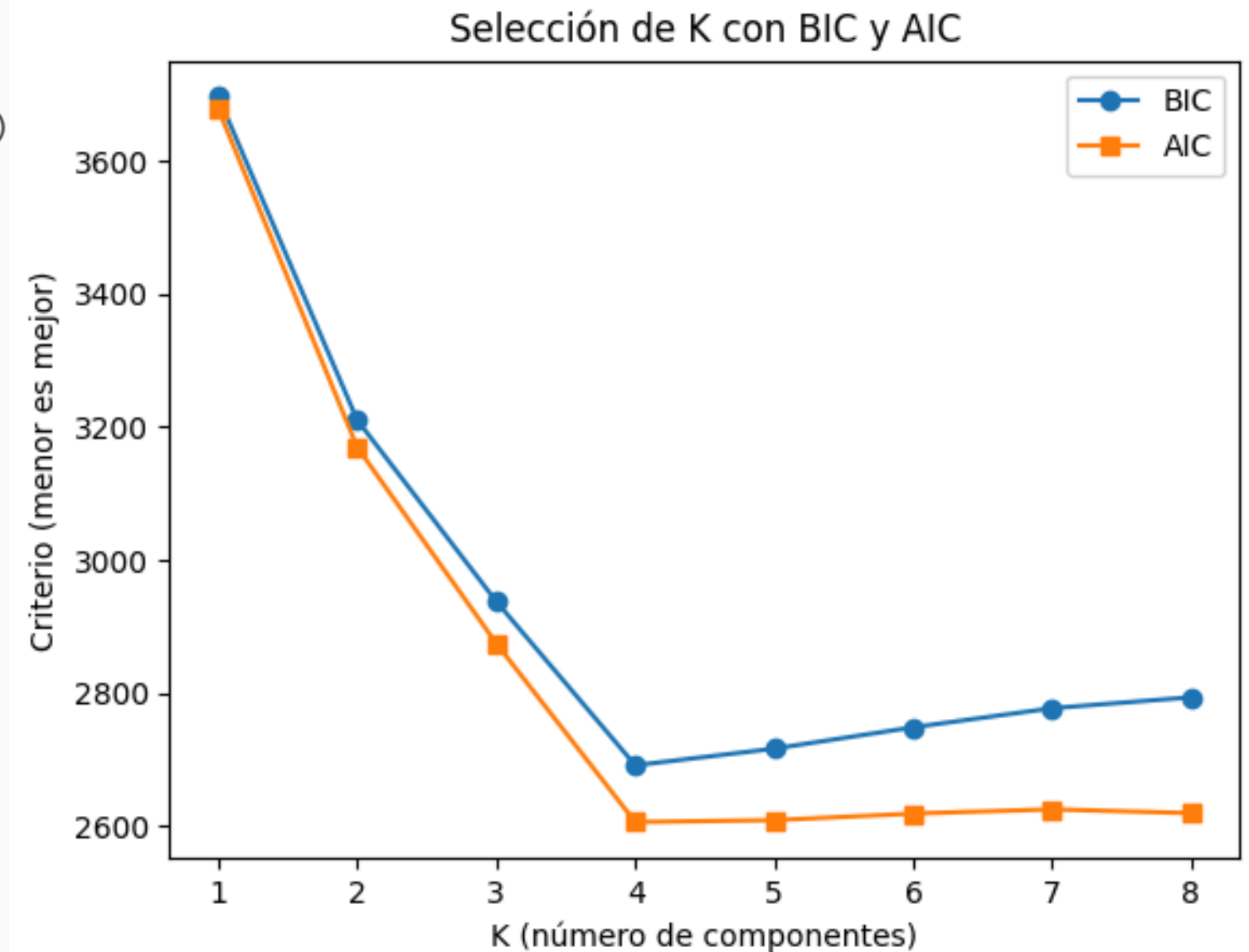
```
from sklearn.mixture import GaussianMixture
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=300, centers=4, cluster_std=[1.3, 0.6, 0.9, 1.6])

# Valores de K
k_range = range(1, 9)
bic_scores = []
aic_scores = []

for k in k_range:
    gmm = GaussianMixture(n_components=k, random_state=42)
    gmm.fit(X)
    bic_scores.append(gmm.bic(X))
    aic_scores.append(gmm.aic(X))

plt.plot(k_range, bic_scores, marker='o', label='BIC')
plt.plot(k_range, aic_scores, marker='s', label='AIC')
plt.xlabel("K (número de componentes)")
plt.ylabel("Criterio (menor es mejor)")
plt.title("Selección de K con BIC y AIC")
plt.legend()
plt.show()
```



# MINERÍA DE DATOS

**Maximiliano Ojeda**

[muojeda@uc.cl](mailto:muojeda@uc.cl)

---



IIC-2433